

rcl_logging_journal

- ROS 2 rcl logging implementation built on the [journald native protocol](#).
- Structured, indexed log records in the Linux system journal.
- `journalctl` becomes the ROS 2 log tool. Zero daemons, zero config.



Demo



Demo: per node filtering

```
export RCL_LOGGING_IMPLEMENTATION=rcl_logging_journal
ros2 run demo_nodes_cpp talker &
ros2 run demo_nodes_py listener &

journalctl -f ROS2_NODE_NAME=listener
journalctl -p warning ROS2_DISTRO=rolling
journalctl ROS2_NODE_NAME=talker -o verbose -n 1
```

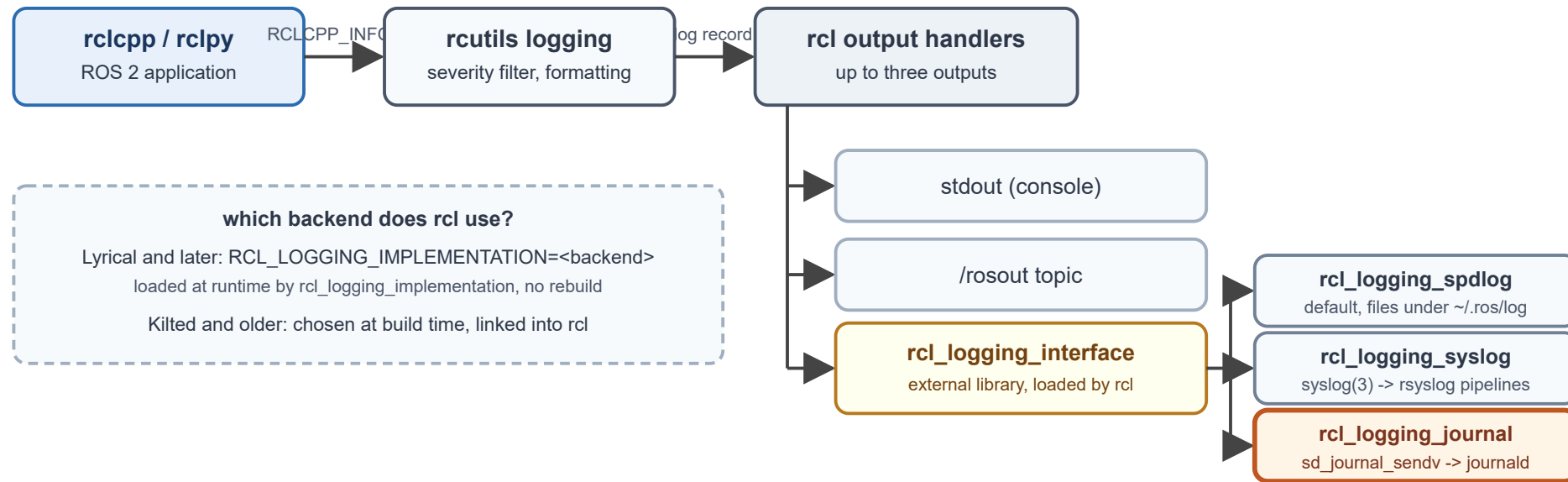
Demo: container to host journal

```
docker run -it --rm \  
  -v /run/systemd/journal/socket:/run/systemd/journal/socket \  
  -e RCL_LOGGING_IMPLEMENTATION=rcl_logging_journal \  
  my_ros2_image ros2 run demo_nodes_cpp talker
```

```
# on the HOST  
journalctl -f ROS2_NODE_NAME=talker
```

- One bind mount, nothing installed or started in the image.
- Host journald owns storage, rotation, retention for every container.
- `_SYSTEMD_CGROUP` / your `CONTAINER=` field tell containers apart.

ROS 2 logging subsystem



What's the Pain?

- ROS 2 writes log **files** in default with spdlog, but ships no log **reader**.
- `~/.ros/log/<exe>_<pid>_<stamp>.log` per process: `grep` , `less` , hand made `logrotate` .
- No "all WARN+ of node X in the last 10 minutes", no "what happened before the reboot".
- Nothing in the file proves *which* process wrote it.

journald in one slide: the protocol

- `AF_UNIX / SOCK_DGRAM` at `/run/systemd/journal/socket` .
- A record is a set of `KEY=VALUE` fields, **not** a text line.
- Too big for a datagram? libsystemd hands journald a sealed `memfd` (zero copy).
- journald attaches trusted fields from the sender's credentials: `_PID _UID _COMM`
`_EXE _CMDLINE _BOOT_ID _HOSTNAME` .

```
MESSAGE=[INFO] [1757236503.620] [talker]: Publishing: 'Hello World: 3'  
PRIORITY=6    ROS2_NODE_NAME=talker  
SYSLOG_IDENTIFIER=talker    ROS2_DISTRO=rolling    ROBOT_ID=amr-07
```

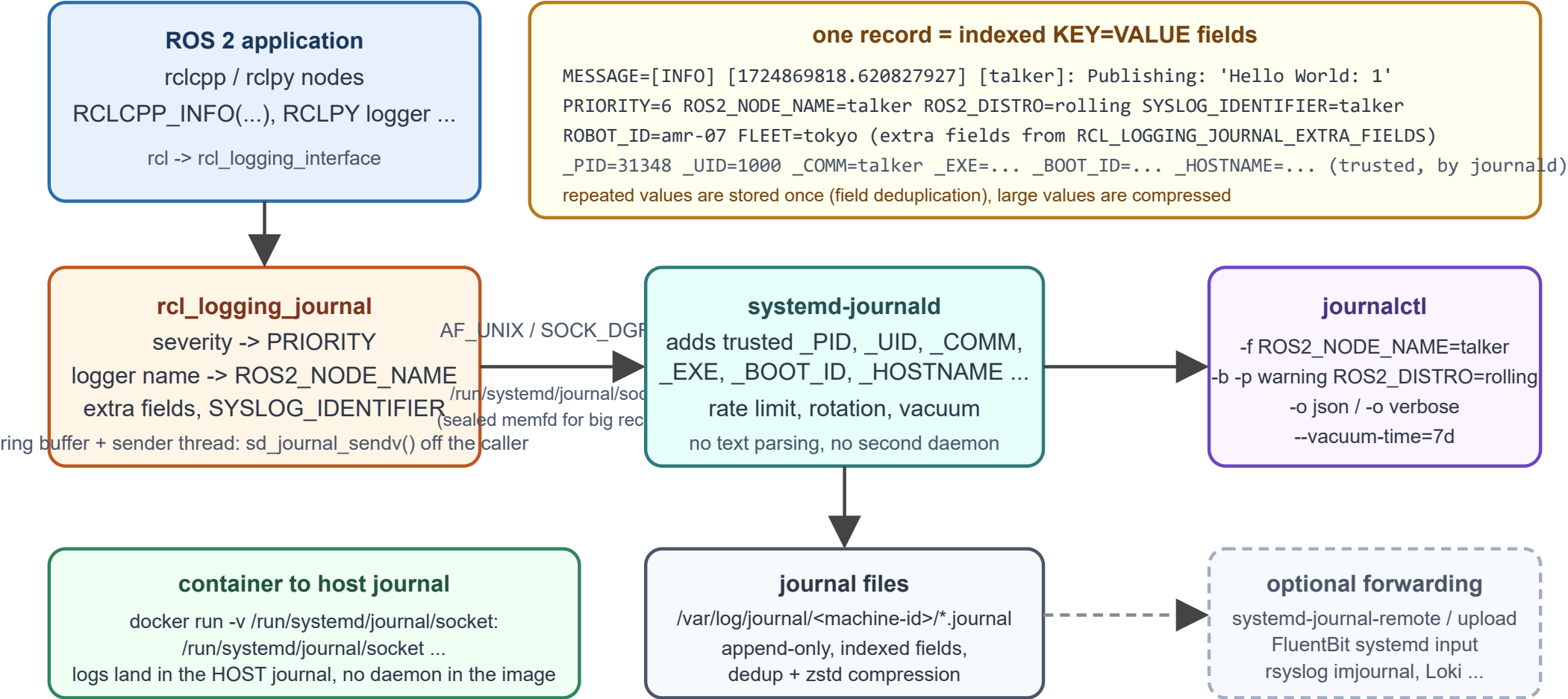
journal in one slide: the files

- Append-only, mmap based, read directly by `journalctl` (no daemon round trip).
- **Field deduplication:** `ROS2_NODE_NAME=talker` , `PRIORITY=6` are stored once, referenced by every entry.
- **Compression** (zstd) for payloads above 512 bytes: backtraces yes, short lines no.
- **Indexed:** `journalctl ROS2_NODE_NAME=talker` walks an index, it does not grep.
- **Rotation, vacuum, retention** built in: `SystemMaxUse=` , `MaxRetentionSec=` ,
`journalctl --vacuum-*` .
- Rotated files stay queryable with the very same command.

journald in one slide: journalctl

```
journalctl -f ROS2_NODE_NAME=talker # tail one node
journalctl -p warning ROS2_DISTRO=rolling --since "10 min ago"
journalctl -b -1 -p err ROS2_DISTRO=rolling # previous boot
journalctl _PID=3141 -o verbose # trusted metadata
journalctl ROS2_NODE_NAME=talker -o json > talker.jsonl # export
journalctl -F ROS2_NODE_NAME # who logged?
sudo journalctl --vacuum-time=7d # retention
```

rcl_logging_journal: ROS 2 logging into the systemd journal



rcl_logging_journal: design

rcl_logging_interface	backend
<code>initialize</code>	resolve identifier, parse env, probe <code>/run/systemd/journal/socket</code> , pre-build constant fields
<code>log(severity, name, msg)</code>	<code>memcpy</code> into a ring buffer (1 MiB default), return; a sender thread does <code>sd_journal_sendv()</code>
<code>set_logger_level</code>	ignored: rcl filters first, journald has <code>MaxLevelStore=</code>
<code>shutdown</code>	drain the ring, report undeliverable count

- FATAL is synchronous: journald fsyncs `CRIT` , so FATAL + crash is on disk.
- Order preserved (one consumer), nothing dropped (full ring blocks), exit drains.

Configuration: env only

variable	default	purpose
<code>RCL_LOGGING_JOURNAL_IDENTIFIER</code>	executable	<code>SYSLOG_IDENTIFIER</code> , <code>journalctl -t</code>
<code>RCL_LOGGING_JOURNAL_EXTRA_FIELDS</code>	empty	<code>ROBOT_ID=amr-07;FLEET=tokyo</code> , indexed
<code>RCL_LOGGING_JOURNAL_STRICT</code>	1	fail init without journald, or no-op
<code>RCL_LOGGING_JOURNAL_BUFFER_SIZE</code>	1M	ring buffer capacity, 4K to 1G

- Rotation, compression, rate limits: `journald.conf(5)` , not duplicated here.
- Shipped drop-in: `config/ros2-journald.conf` .

Issues and PRs always welcome 🚀

https://github.com/fujitatomoya/rcl_logging_journal

- [design.md](#)
- [tutorials](#)

